Reinforcement Learning

Clase 13

Ignacio Brevis

ignaciobrve@unisabana.edu.co

Universidad de La Sabana

11 noviembre 2026



Repaso: Policy gradient methods

En los casos anteriores, representamos las políticas en forma de tablas, donde podíamos cambiar la política de un estado independientemente de los otros estados

	a_1	a_2		a_m
s_1	$\pi(a_1\|s_1)$	$\pi(a_2\|s_1)$	$\dots$	$\pi(a_m\|s_1)$
s_2	$\pi(a_1\|s_2)$	$\pi(a_2\|s_2)$	$\dots$	$\pi(a_m\|s_2)$
$\vdots$	$\vdots$	$\vdots$		$\vdots$
s_n	$\pi(a_1\|s_n)$	$\pi(a_2\|s_n)$	$\dots$	$\pi(a_m\|s_n)$

value-based methods

$\downarrow$

policy-based methods

$$\boxed{\pi(a|s;\theta)}$$

$\theta \in \mathbb{R}^d$ son los parámetros a entrenar

Repaso: Policy gradient methods (Metrics)

Métrica	suma	valor esperado	límite
$\bar{v}_\pi$	$\sum_{s\in\mathcal{S}} d(s) v_\pi(s)$	$\mathbb{E}_{S\sim d}[v_\pi(S)]$	$\lim_{n\to\infty} \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R_{t+1}\right]$
$\bar{r}_\pi$	$\sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s)$	$\mathbb{E}_{S\sim d_\pi}[r_\pi(S)]$	$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right]$

Average state value

Average reward.

$\nabla_\theta \bar{v}_\pi$?

$\nabla_\theta \bar{r}_\pi$?

- Estas métricas son funciones de π (por lo tanto, de la parametrización θ).
- Las funciones de pérdida $J(\theta)$ estarán definidas en términos de estas métricas. Entonces, nuestro problema se traduce en **buscar θ que maximice estas métricas**.
- Calcular los gradientes es una de las partes más complejas de los métodos de policy gradient, por lo que debemos distinguir varios casos!

Repaso: Policy gradient methods (Gradients)

Caso con descuento ($\gamma < 1$)

Después de algunas cuentas...

$$\nabla_\theta v_\pi(s) = \sum_{s' \in \mathcal{S}} \Big[(I_n - \gamma P_\pi)^{-1} \Big]_{ss'} \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s';\theta) q_\pi(s', a),$$

donde $[(I_n - \gamma P_\pi)^{-1}]_{ss'} = \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{ss'}$.

- $[P_\pi]_{ss'}$ es la probabilidad de transición de s a s' (en un paso).
- $[P_\pi^k]_{ss'}$ es la probabilidad de transición de con k pasos.
- $\Pr_\pi(s'|s) := [(I_n - \gamma P_\pi)^{-1}]_{ss'}$ es la probabilidad de transición con cualquier número de pasos.

Detalles en Zhao, Mathematical foundations of RL, Box 9.2: Proof of Lemma 9.2

Repaso: Policy gradient methods (Gradients)

$$\bar{v}_\pi := \mathbb{E}_{S\sim d}\Big[v_\pi(S)\Big] = \sum_{s\in\mathcal{S}} d(s) v_\pi(s) \quad \text{(average state value)}$$

Caso cuando $d = d_0$ (independiente de la política)

Theorem (Gradiente de $\bar{v}_\pi^0$)

En el caso donde $\gamma \in (0,1)$ *y* $d = d_0$ *es independiente de la política, el gradiente de* $\bar{v}_\pi := \bar{v}_\pi^0$ *viene dado por:*

$$\nabla_\theta \bar{v}_\pi^0 = \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big],$$

donde (para el gradiente) $S \sim \rho_\pi$ *y* $A \sim \pi(S;\theta)$. *Aquí* ρ_π *denota:*

$$\rho_\pi(s) = \sum_{s'\in\mathcal{S}} d_0(s') \Pr_\pi(s|s'), \quad s \in \mathcal{S}.$$

Repaso: Policy gradient methods (Gradients)

$$\boxed{\bar{v}_\pi := \mathbb{E}_{S\sim d}\Big[v_\pi(S)\Big] = \sum_{s\in\mathcal{S}} d(s) v_\pi(s)} \quad \text{(average state value)}$$

Caso cuando $d = d_\pi$ (distribución estacionaria)

Theorem (Gradiente de $\bar{v}_\pi$)

En el caso donde $\gamma \in (0,1)$ *y* $d = d_\pi$, *el gradiente de* $\bar{v}_\pi$ *viene dado por:*

$$\begin{aligned}\nabla_\theta \bar{v}_\pi &\approx \frac{1}{1-\gamma} \sum_{s\in\mathcal{S}} d_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s;\theta) q_\pi(s,a) \\ &= \frac{1}{1-\gamma} \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big],\end{aligned}$$

donde $S \sim d_\pi$ *y* $A \sim \pi(S;\theta)$.

Repaso: Policy gradient methods (Undiscounted case)

Caso sin descuento ($\gamma = 1$)

- Redefinimos los valores de estado y de acciones:

$$v_\pi(s) = \mathbb{E}\Big[(R_{t+1} - \bar{r}_\pi) + (R_{t+2} - \bar{r}_\pi) + \ldots | S_t = s\Big],$$

$$q_\pi(s) = \mathbb{E}\Big[(R_{t+1} - \bar{r}_\pi) + (R_{t+2} - \bar{r}_\pi) + \ldots | S_t = s, A_t = a\Big].$$

- La ecuación de Bellman es "reemplazada" por la ecuación de Poisson

$$v_\pi = r_\pi - \bar{r}_\pi \mathbf{1}_n + P_\pi v_\pi.$$

- El gradiente viene dado por:

$$\nabla_\theta \bar{r}_\pi = \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big],$$

donde $S \sim d_\pi$ y $A \sim \pi(S;\theta)$.

Detalles en Zhao, Mathematical foundations of RL, Section 9.3.2

Repaso: Policy gradient methods (Summary)

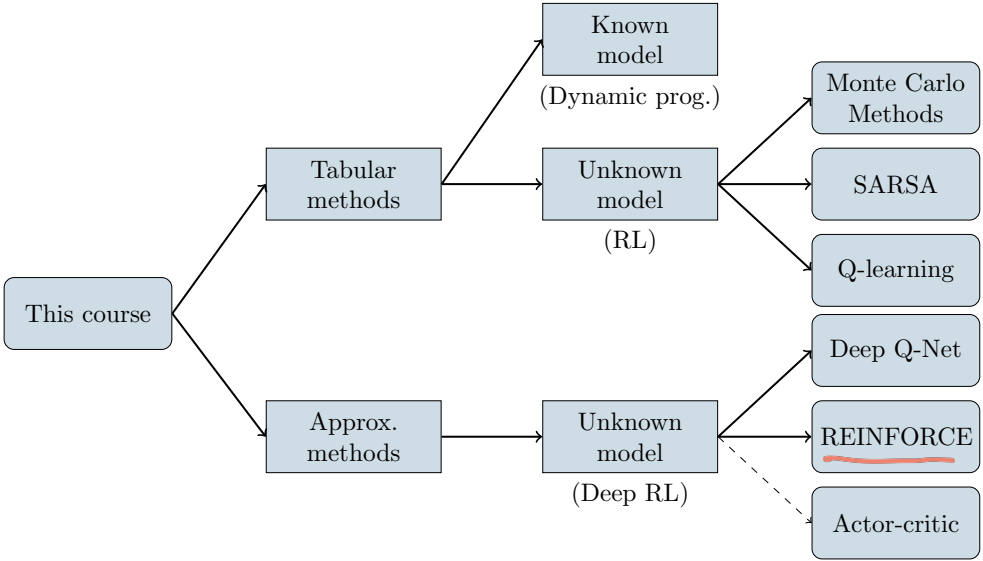
Resumen:

	Discounted	Undiscounted
$\bar{v}_\pi$ defined?	Yes, unique	Non-unique
$\bar{r}_\pi$ defined?	Yes	Yes
$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi$	Holds ✔	Breaks down ✗
Policy gradient of $\bar{r}_\pi$	Approximate	Exact
State values satisfy	Bellman equation	Poisson equation

Para maximizar $J(\theta)$ utilizaremos gradient ascent. En todos los casos, podemos asumir que gradiente viene dado por:

$$\nabla_\theta J(\theta) \text{ “} = \text{” } \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big]$$

Contenido


This course
Tabular methods
Approx. methods
Known model
(Dynamic prog.)
Unknown model
(RL)
Unknown model
(Deep RL)
Monte Carlo Methods
SARSA
Q-learning
Deep Q-Net
REINFORCE
Actor-critic

Policy gradient methods: REINFORCE

Daddo $J(\theta)$ por alguna de las métricas. El primer método de policy gradient puede ser derivado de la siguiente forma:

G.A.

$$\theta_{t+1} = \theta_t + \alpha \mathbb{E}\Big[\nabla_\theta \ln \pi(A \mid S; \theta_t) q_\pi(S, A)\Big]$$

$\alpha \nabla J(\theta)$

$\nabla J(\theta)$

Cómo estimamos $q_\pi(S,A)$?

MC → REINFORCE

TD → Actor-Critic

Machine Learning, 8, 229-256 (1992)
© 1992 Kluwer Academic Publishers, Boston. Manufactured in The Netherlands.

Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning

RONALD J. WILLIAMS rjw@corwin.ccs.northeastern.edu
College of Computer Science, 161 CN, Northeastern University, 360 Huntington Ave., Boston, MA 02115

Abstract. This article presents a general class of associative reinforcement learning algorithms for connectionist networks containing stochastic units. These algorithms, called REINFORCE algorithms, are shown to make weight adjustments in a direction that lies along the gradient of expected reinforcement in both immediate-reinforcement tasks and certain limited forms of delayed-reinforcement tasks, and they do this without explicitly computing gradient estimates or even storing information from which such estimates could be computed. Specific examples of such algorithms are presented, some of which bear a close relationship to certain existing algorithms while others are novel but potentially interesting in their own right. Also given are results that show how such algorithms can be naturally integrated with backpropagation. We close with a brief discussion of a number of additional issues surrounding the use of such algorithms, including what is known about their limiting behaviors as well as further considerations that might be used to help develop similar but potentially more powerful reinforcement learning algorithms.

Keywords. Reinforcement learning, connectionist networks, gradient descent, mathematical analysis

Policy gradient methods: REINFORCE

$$\mathbb{E}\left(\nabla_\theta \ln \pi(A|S;\theta)\, q_\pi(S,A)\right)$$

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t;\theta_t) q_t(s_t,a_t)$$

↓ approx. estoc.

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t;\theta_t) q_t(s_t,a_t)$$

$$\nabla \ln(\pi) = \frac{\nabla \pi}{\pi}$$

↓ reescribir

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t,a_t)}{\pi(a_t|s_t;\theta_t)}\right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t;\theta_t)$$

¿ β_t ?

Policy gradient methods: REINFORCE

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t, a_t)}{\pi(a_t|s_t;\theta_t)} \right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t;\theta_t)$$

- Si $\beta_t < 0$ $\Longrightarrow q_t(s_t, a_t) < 0$

  $\pi(a_t|s_t;\theta_{t+1}) < \pi(a_t|s_t;\theta_t)$

- Si $\beta_t > 0$ $\Longrightarrow q_t(s_t, a_t) > 0$

  $\pi(a_t|s_t;\theta_{t+1}) > \pi(a_t|s_t;\theta_t)$

Policy gradient methods: REINFORCE

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t, a_t)}{\pi(a_t|s_t;\theta_t)} \right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t;\theta_t)$$

- β_t es proporcional a $q_t(s_t, a_t)$

$\beta_t = \frac{q_t}{\pi}$ Si q_t dismina, aumenta la prob. de tomar a_t para s_t.

$\Rightarrow$ aumenta la Explotación

- β_t es inversamente proporcional a $\pi(a_t|s_t;\theta_t)$

Si $\pi(a_t|s_t;\theta_t)$ disminuye $\Rightarrow$ β_t pequeño $\Rightarrow$ θ_{t+1} cambia "poco"

$\Rightarrow$ Aumenta la Exploración

Policy gradient methods: REINFORCE

Obs: $\beta_t = \dfrac{q_t(s_t, a_t)}{\pi(a_t|s_t;\theta_t)}$ nos da un balance entre explorar y explotar.

Volviendo a la expresión original:

$$\mathbb{E}_{S\sim\eta, A\sim\pi}\Big[\nabla_\theta \ln \pi(A \mid S;\theta_t) q_\pi(S,A)\Big]$$

- $S \sim \eta$, donde η es una distribución dada por ρ_π (cuando d es independiente de la política) o d_π (cuando d toma la distribución estacionaria).
- A debe seguir la distribución dada por la política $\pi(A|S;\theta)$, por lo que es un método on-policy.
- $q_\pi(S,A)$ es aproximado por $q_t(s_t,a_t) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$ (Monte Carlo)

Policy gradient methods: REINFORCE

REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility

Algorithm 9.1: Policy Gradient by Monte Carlo (REINFORCE)

Initialization: Initial parameter θ; $\gamma \in (0, 1)$; $\alpha > 0$.
Goal: Learn an optimal policy for maximizing $J(\theta)$.

For each episode, do
- Generate an episode $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi(\theta)$.
  For $t = 0, 1, \ldots, T-1$:
  Value update: $q_t(s_t, a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
  Policy update: $\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta) q_t(s_t, a_t)$

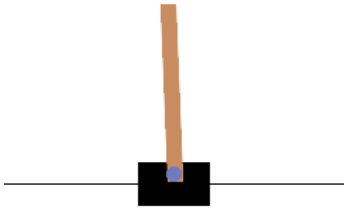
Page 212, Zhao, Mathematical foundations of reinforcement learning

REINFORCE: Example

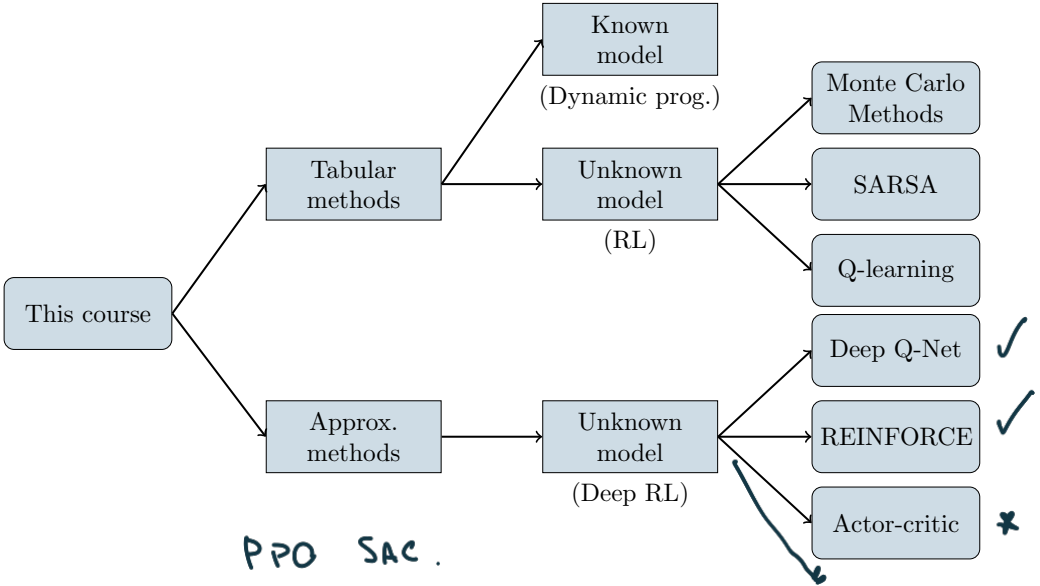
See `https://gymnasium.farama.org/environments/classic_control/cart_pole/` for a guided introduction to the environment

In `https://github.com/ibrevis/RL_unisabana_2026_2`

- go to `/Lesson13/`
- Go `REINFORCE_CartPole.ipynb` and tune the hyperparameters to traing the policy.
- Check your results with `run_reinforce_cartpole_policy.py`.



Contenido


This course
Tabular methods
Approx. methods
Known model
(Dynamic prog.)
Unknown model
(RL)
Unknown model
(Deep RL)
Monte Carlo Methods
SARSA
Q-learning
Deep Q-Net
REINFORCE
Actor-critic
PPO SAC.

Policy gradient methods: Actor-Critic

$$\theta_{t+1} = \theta_t + \alpha \mathbb{E}\Big[\nabla_\theta \ln \pi(A \mid S; \theta_t) q_\pi(S, A)\Big] \quad *$$

↓ aprox. estoc.

$$\boxed{\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t | s_t; \theta_t) q_t(s_t, a_t)}$$

MC → REINFORCE

TD → Actor-critic

Estimación de q-values

- Queremos estimar $q_t(s_t, a_t)$ mediante $q(s_t, a_t; w_t)$
- SARSA nos permite estimar el **valor de la acción** por medio del algoritmo de TD:

$$w_{t+1} = w_t + \alpha\Big(r_{t+1} + \gamma q(s_{t+1}, a_{t+1}; w_t) - q(s_t, a_t; w_t)\Big)\nabla_w q(s_t, a_t; w_t)$$

- En cada paso t requiere $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}$
- A continuación presentaremos la versión mas simple de los métodos actor-critic: Q actor-critic (QAC).

Policy gradient methods: Actor-Critic

- **Actor**: se refiere a **la política** y tu actualización (la política dicta las acciones)



$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t|s_t;\theta_t) q_t(s_t, a_t; w_t)$$

- **Critic**: se refiere a la **evaluación** de la política por medio de una función de valor (critica la performance the la política)



$$w_{t+1} = w_t + \alpha_w \Big(r_{t+1} + \gamma q(s_{t+1}, a_{t+1}; w_t) - q(s_t, a_t; w_t) \Big) \nabla_w q(s_t, a_t; w_t)$$

Policy gradient methods: Actor-Critic

Simplest actor-critic algorithm:

Algorithm 10.1: The simplest actor-critic algorithm (QAC)

Initialization: A policy function $\pi(a|s,\theta_0)$ where θ_0 is the initial parameter. A value function $q(s,a,w_0)$ where w_0 is the initial parameter. $\alpha_w, \alpha_\theta > 0$.
Goal: Learn an optimal policy to maximize $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t,\theta_t)$, observe r_{t+1}, s_{t+1}, and then generate a_{t+1} following $\pi(a|s_{t+1},\theta_t)$.

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q(s_t,a_t,w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \big[r_{t+1} + \gamma q(s_{t+1},a_{t+1},w_t) - q(s_t,a_t,w_t)\big] \nabla_w q(s_t,a_t,w_t)$$

Page 217, Zhao, Mathematical foundations of reinforcement learning

Policy gradient methods: Actor-Critic

Ventajas:

- Menor varianza que en REINFORCE.
- Online e incremental.
- Combina métodos policy-based y value-based.

Desventajas:

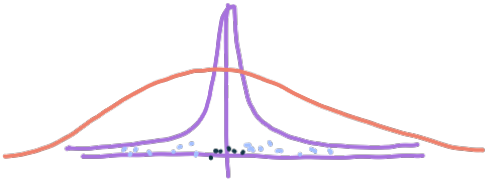
- Introduce bias.
- Alta varianza en la actualización del actor.
- On-policy.
- Dos redes neuronales que entrenar.

Policy gradient methods: Actor-Critic

Llamemos

$$X(S, A) = \nabla_\theta \ln \pi(A \mid S; \theta_t) q_\pi(S, A)$$

- Queremos calcular $\mathbb{E}[X(S, A)]$ por medio de muestras (estocásticas)
- Para tener mejores estimaciones, la varianza de X tiene que ser pequeña.



Notemos que

$$\mathbb{E}_{S\sim d, A\sim\pi}\left[\nabla_\theta \ln \pi(A \mid S;\theta_t) q_\pi(S,A)\right] = \mathbb{E}_{S\sim d, A\sim\pi}\left[\nabla_\theta \ln \pi(A \mid S;\theta_t)(q_\pi(S,A) - b(S))\right]$$

$$\mathbb{E}_{S\sim d, A\sim\pi}\left(\nabla_\theta \ln \pi(A|S;\theta) b(S)\right) = 0$$

$$= \sum_s d(s) \sum_a \pi(a|s;\theta) \nabla \ln \pi(a|s;\theta) b(s)$$

$$= \sum_s d(s) \sum_a \pi(a|s;\theta) \frac{\nabla \pi(a|s;\theta)}{\pi(a|s,\theta)} b(s)$$

$$= \sum_s d(s) \sum_a b(s) \nabla \pi(a|s;\theta)$$

Notemos que (continuación...)

baseline

$$\mathbb{E}_{S\sim d, A\sim\pi}\Big[\nabla_\theta \ln \pi(A \mid S;\theta_t) q_\pi(S,A)\Big] = \mathbb{E}_{S\sim d, A\sim\pi}\Big[\nabla_\theta \ln \pi(A \mid S;\theta_t)(q_\pi(S,A) - b(S))\Big]$$

$$= \sum_s d(s) \sum_a b(s) \nabla \pi(a|s;\theta)$$

$$= \sum_s d(s) b(s) \sum_a \nabla \pi(a|s;\theta) = \sum_s d(s) b(s) \nabla \sum_a \pi(a|s;\theta)$$

$$= \sum_s d(s) b(s) \nabla(1) = 0$$

b no puede depender de A (solo de S)

Policy gradient methods: Actor-Critic

Ahora llamemos

$$X(S, A) = \nabla_\theta \ln \pi(A \mid S; \theta_t)(q_\pi(S, A) - b(S))$$

(hemos agregado $b(S)$)

- $b(S)$ se conoce como función **baseline**.
- $\mathbb{E}[X(S, A)]$ no depende de $b(S)$ (siempre y cuando b no dependa de A).
- $\text{var}(X)$ no es invariante a $b(S)$.
- Queremos encontrar $b(S)$ tal que minimice $\text{var}(X)$.

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}(\|\nabla_\theta \ln \pi\|^2 q_\pi(s, A))}{\mathbb{E}_{A\sim\pi}(\|\nabla_\theta \ln \pi\|^2)}$$

función baseline optimal:

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}\left[\|\nabla_\theta \ln \pi(A|s;\theta_t)\|^2 q_\pi(s,A)\right]}{\mathbb{E}_{A\sim\pi}\left[\|\nabla_\theta \ln \pi(A|s;\theta_t)\|^2\right]}$$

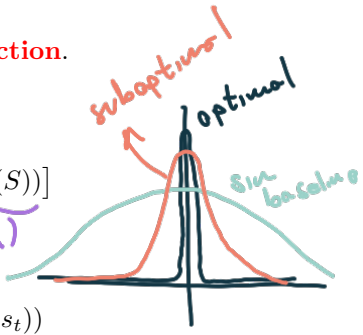
- A pesar de ser optimal, es muy complejo de calcular como para ser útil.
- Notemos que $b^*(s)$ es un promedio con peso de $q_\pi(s,A)$ (sobre las acciones).
- El peso utilizado para cada acción a es $\|\nabla_\theta \ln \pi(a|s;\theta_t)\|^2$.

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}(1 \cdot q_\pi(s,A))}{\mathbb{E}_{A\sim\pi}(1)} = \mathbb{E}(q_\pi(s,A))$$

función baseline suboptimal:

$$b^\dagger(s) = \mathbb{E}_{A\sim\pi}\big[q_\pi(s,A)\big] = v_\pi(s)$$

- $b^\dagger(s)$ es mucho más simple y proporciona una reducción significativa de la varianza (ya que resta el valor promedio, independiente de la acción).
- $\delta_\pi(S,A) := q_\pi(S,A) - v_\pi(S)$ se conoce como **advantage function**.



Nuevo (gradient-ascent) algoritmo:

$$\theta_{t+1} = \theta_t + \alpha\mathbb{E}\big[\nabla_\theta \ln \pi(A|S;\theta_t)(q_\pi(S,A) - v_\pi(S))\big]$$

$\delta_\pi(S,A)$

Versión estocástica:

$$\theta_{t+1} = \theta_t + \alpha\nabla_\theta \ln \pi(a_t|s_t;\theta_t)(q_\pi(s_t,a_t) - v_\pi(s_t))$$

Policy gradient methods: Actor-Critic

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t;\theta_t)(q_\pi(s_t,a_t) - v_\pi(s_t))$$

$$= \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t;\theta_t)\delta_\pi(s_t,a_t)$$

$$= \theta_t + \alpha \frac{\nabla_\theta \pi(a_t|s_t;\theta_t)}{\pi(a_t|s_t;\theta_t)}\delta_\pi(s_t,a_t)$$

$$= \theta_t + \alpha \left(\frac{\delta_\pi(s_t,a_t)}{\pi(a_t|s_t;\theta_t)} \right) \nabla_\theta \pi(a_t|s_t;\theta_t)$$

Explorar

Explotar

Policy gradient methods: Actor-Critic

Ahora tenemos:

$$\pi(A|S;\theta), \quad q_\pi(S,A;w), \quad \text{y} \quad v_\pi(S)$$

qué hacemos?

vemos que

$$\mathbb{E}\left(q_\pi(S,A) - v_\pi(S) \mid S=s_t, A=a_t\right)$$

$$= \mathbb{E}\left(R + \gamma v_\pi(S') - v_\pi(S) \mid S=s_t, A=a_t\right)$$

$$\delta_t = q_t(s_t,a_t) - v_\pi(s_t)$$

$$\downarrow$$

$$r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t)$$

Policy gradient methods: Actor-Critic

Advantage Actor-Critic method (A2C):

Algorithm 10.2: Advantage actor-critic (A2C) or TD actor-critic

Initialization: A policy function $\pi(a|s,\theta_0)$ where θ_0 is the initial parameter. A value function $v(s,w_0)$ where w_0 is the initial parameter. $\alpha_w, \alpha_\theta > 0$.
Goal: Learn an optimal policy to maximize $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t,\theta_t)$ and then observe r_{t+1}, s_{t+1}.

Advantage (TD error):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1},w_t) - v(s_t,w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t,w_t)$$

Page 221, Zhao, Mathematical foundations of reinforcement learning